

Name: Anvith Shetty

PRN: 202401100138

BATCH : A4

AIML MDM LAB

Practical Assignment 1: PEAS Descriptors of Task Environment

`:- dynamic dirty/1.`

`:- dynamic vacuum_location/1.`

`room(a).`

`room(b).`

`room(c).`

`adjacent(a, b).`

`adjacent(b, a).`

`adjacent(b, c).`

`adjacent(c, b).`

`dirty(a).`

`dirty(c).`

`vacuum_location(a).`

```
action(clean) :-
```

```
    vacuum_location(Room),  
    dirty(Room), !.
```

```
action(move(b)) :-
```

```
    vacuum_location(a),  
    dirty(c), !.
```

```
action(move(c)) :-
```

```
    vacuum_location(b),  
    dirty(c), !.
```

```
action(stop) :-
```

```
    \+ dirty(_).
```

```
perform(clean) :-
```

```
    vacuum_location(Room),  
    retract(dirty(Room)),  
    format("Vacuum cleaned room ~w.~n", [Room]).
```

```
perform(move(ToRoom)) :-
```

```
    retract(vacuum_location(CurrentRoom)),  
    assert(vacuum_location(ToRoom)),  
    format("Vacuum moved from room ~w to room ~w.~n",  
          [CurrentRoom, ToRoom]).
```

```
perform(stop) :-
```

```
    format("All rooms are clean. Stopping...~n", []).
```

```
start :-
```

```
    action(Action),
```

```
    format("Selected action: ~w~n", [Action]),
```

```
    perform(Action),
```

```
    (Action == stop -> true ; start).
```

```
reset :-
```

```
    retractall(dirty(_)),
```

```
    retractall(vacuum_location(_)),
```

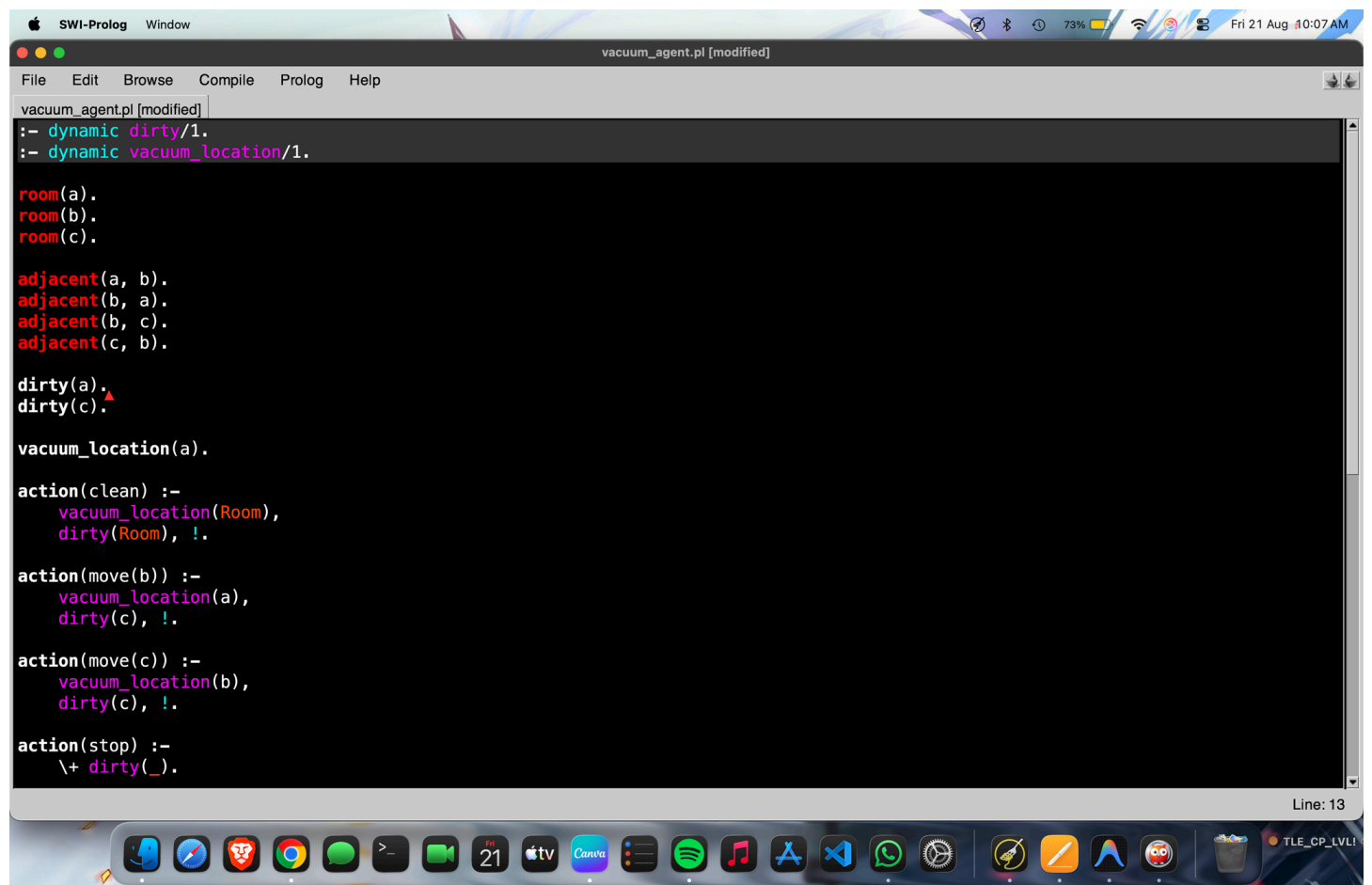
```
    assert(dirty(a)),
```

```
    assert(dirty(c)),
```

```
    assert(vacuum_location(a)),
```

```
    format("Simulation reset.~n", []).
```

OUTPUT:



The screenshot shows a Prolog IDE window titled "vacuum_agent.pl [modified]". The code defines a vacuum agent simulation with three rooms (a, b, c) and their adjacencies. It includes predicates for room status (dirty/clean) and vacuum location. The initial state is defined by dynamic facts and initial room states.

```
vacuum_agent.pl [modified]
:- dynamic dirty/1.
:- dynamic vacuum_location/1.

room(a).
room(b).
room(c).

adjacent(a, b).
adjacent(b, a).
adjacent(b, c).
adjacent(c, b).

dirty(a).
dirty(c).

vacuum_location(a).

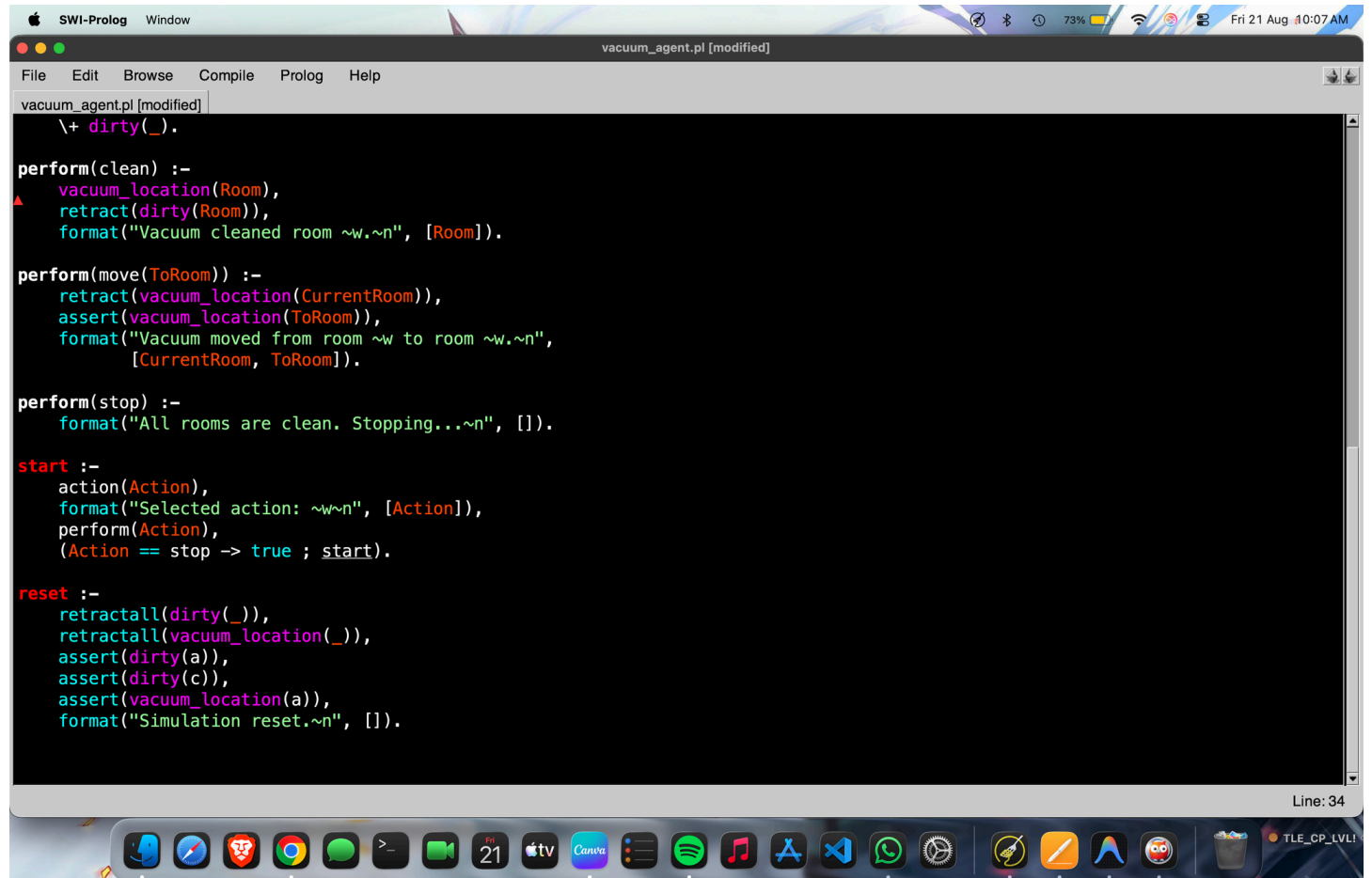
action(clean) :-
    vacuum_location(Room),
    dirty(Room), !.

action(move(b)) :-
    vacuum_location(a),
    dirty(c), !.

action(move(c)) :-
    vacuum_location(b),
    dirty(c), !.

action(stop) :-
    \+ dirty(_).
```

Line: 13



The screenshot shows the continuation of the Prolog code, defining the perform actions, start, and reset predicates. The perform actions handle cleaning, moving, and stopping. The start predicate initiates the simulation, and the reset predicate resets the state.

```
\+ dirty(_).

perform(clean) :-
    vacuum_location(Room),
    retract(dirty(Room)),
    format("Vacuum cleaned room ~w.~n", [Room]).

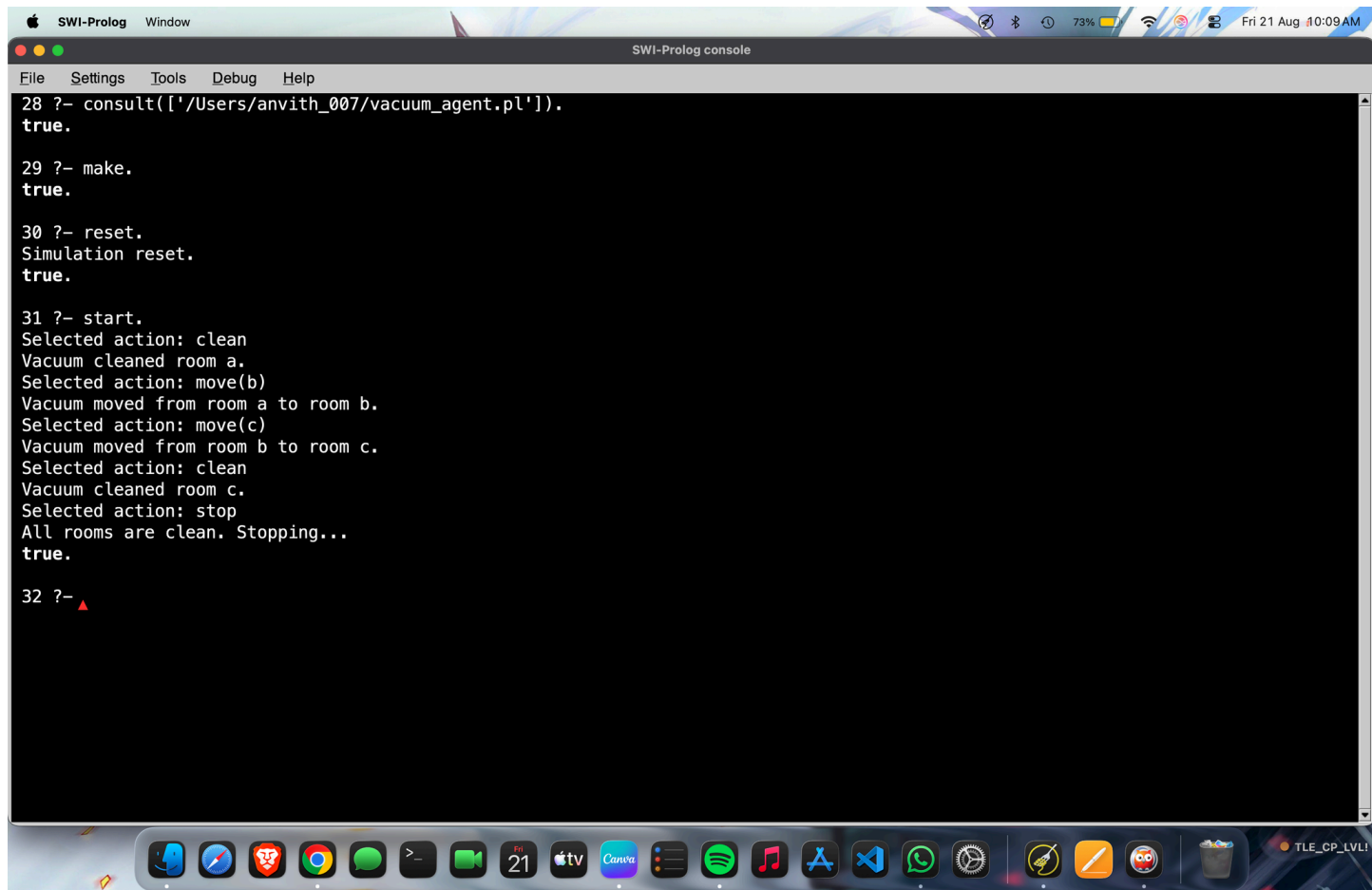
perform(move(ToRoom)) :-
    retract(vacuum_location(CurrentRoom)),
    assert(vacuum_location(ToRoom)),
    format("Vacuum moved from room ~w to room ~w.~n",
        [CurrentRoom, ToRoom]).

perform(stop) :-
    format("All rooms are clean. Stopping...~n", []).

start :-
    action(Action),
    format("Selected action: ~w~n", [Action]),
    perform(Action),
    (Action == stop -> true ; start).

reset :-
    retractall(dirty(_)),
    retractall(vacuum_location(_)),
    assert(dirty(a)),
    assert(dirty(c)),
    assert(vacuum_location(a)),
    format("Simulation reset.~n", []).
```

Line: 34



Result:

The vacuum cleaner successfully cleaned all dirty rooms. It cleaned room A, moved from A to B, moved from B to C, cleaned room C, and stopped after all rooms became clean.

Your code itself is present and structured correctly in the PDF. The start and reset procedures are also included.